

TALLER 3 HTML

1. Variables CSS (Custom Properties) y el Ámbito (Scope) Investiga qué son las variables en CSS, cómo se declaran en el selector:root y qué significa que tengan un "alcance global" o "alcance local". ¿Por qué las variables CSS son superiores a las variables de los preprocesadores (como SASS) en términos de manipulación dinámica en el navegador?

R/= - **Variables:** Son contenedores para guardar datos que repites mucho. En vez de escribir un código de color como #3498db en 50 partes distintas, lo guardas con un nombre fácil como --color-marca: #3498db; y lo usas llamándolo así: var(--color-marca). Si el cliente te pide cambiar el color, solo lo editas una vez en el contenedor.

- **Alcance Global (: root):** Imagina que estás en un edificio de apartamentos. El :root es como el **Wi-Fi de la recepción**. Como está en la base de todo el lugar, **cualquier persona de cualquier piso** puede conectarse y usarlo. En CSS, al poner la variable en :root, cualquier botón, texto o sección de tu página puede usar ese color.

- **Alcance Local:** Es como la **red de Wi-Fi privada** que tú pagas dentro de tu propio apartamento. Solo las personas que están dentro de tu casa pueden usarla; el vecino del cuarto piso ni se entera de que existe. En CSS, si creas una variable dentro de la clase. tarjeta, solo los elementos que estén metidos dentro de esa tarjeta podrán usarla.

- **Dinamismo vs. SASS:** Las variables de SASS se "congelan" (se vuelven texto fijo) cuando guardas tu código y se compila. En cambio, las variables CSS nativas se quedan "vivas" en el navegador. Esto significa que puedes usar JavaScript para cambiarlas en tiempo real mientras el usuario navega (por ejemplo, para cambiar los colores de toda la página al activar un botón de "Modo Oscuro").

2. Arquitectura y Modularización del CSS Cuando un proyecto web tiene decenas de páginas, un solo archivo style.css de 3000 líneas se vuelve obsoleto. Investiga en qué consiste la estrategia de dividir el CSS en múltiples archivos (ej. base.css, layout.css, components.css) y cómo se utiliza la regla @import o las etiquetas múltiples para unificarlos.

R/= - **La Estrategia:** Imagina que escribir todo tu CSS en un solo archivo es como meter toda la ropa de tu casa (camisas, zapatos, ropa interior, abrigos) en una sola caja gigantesca. Encontrar algo ahí dentro se vuelve una pesadilla. Modularizar es **crear cajones ordenados**: pones las reglas generales en un cajón (base.css), la estructura de las habitaciones en otro (layout.css) y las piezas pequeñas como botones o tarjetas en otro (components.css).

- **¿Cómo se unifican?** El navegador necesita leer un solo mapa final. Para juntar todos tus "cajones", tienes dos opciones nativas:

- **Etiquetas <link> múltiples:** Es ir al HTML y llamar a cada archivo por separado, uno detrás de otro. Es como contratar a tres repartidores distintos para que traigan cada caja.
- **La regla @import:** Creas un archivo maestro (por ejemplo, main.css) y adentro escribes líneas como @import 'base.css'; Es como tener una caja principal que adentro tiene notas diciendo: *"ve a buscar la información a la otra caja"*.

3. Mapa de Herencia y Cascada de Variables CSS Crea un esquema jerárquico (un árbol de componentes HTML) donde muestres visualmente cómo una variable CSS declarada en el :root es heredada por un contenedor padre, pero es "sobrescrita" localmente por un contenedor hijo, cambiando el estilo de sus elementos internos.

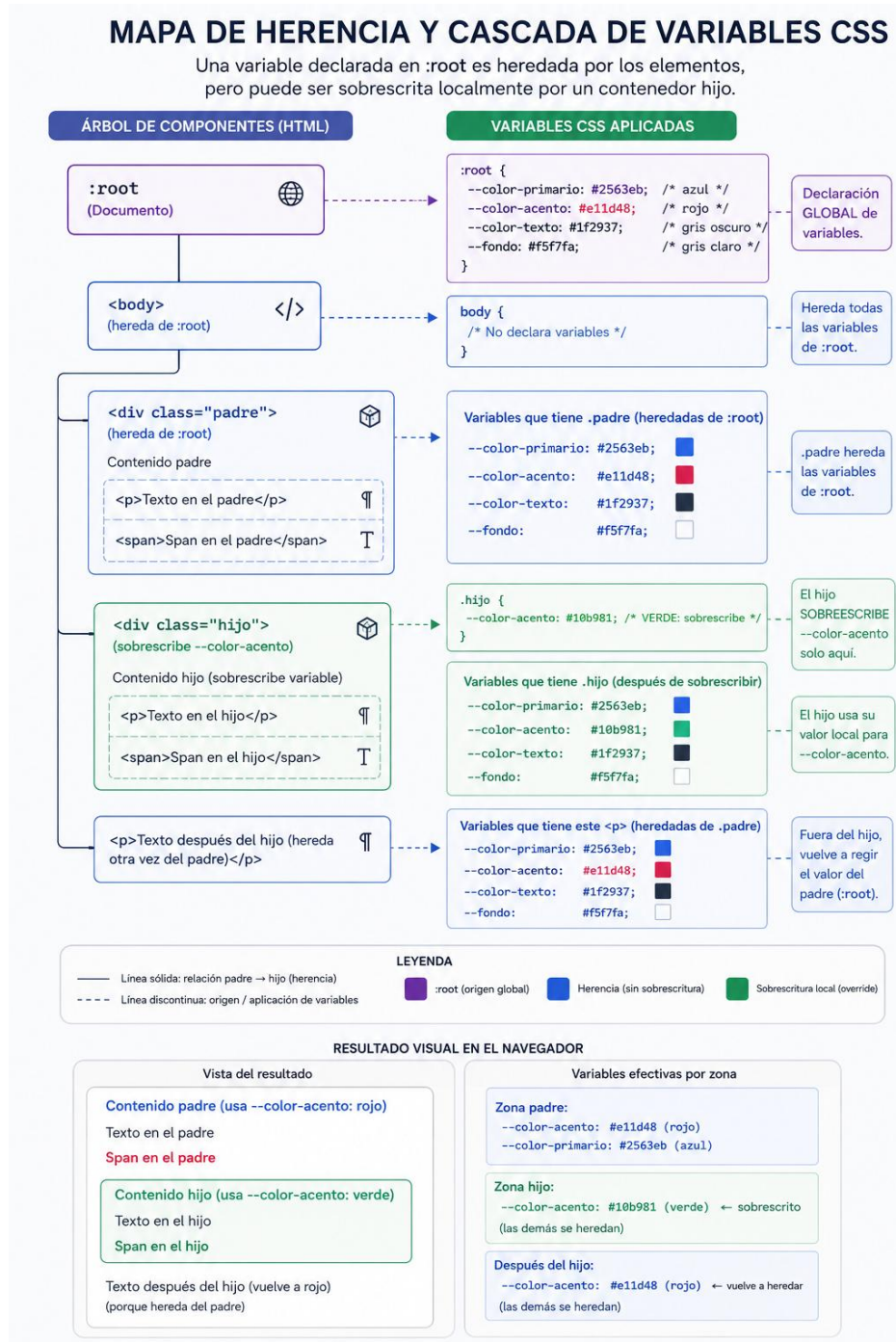
R/= - Transiciones (transition): Consisten en ir directo y de forma suave del Punto A al Punto B. Tienen un inicio y un final definidos. Necesitan obligatoriamente un detonante o una acción del usuario en la página para activarse, como pasar el cursor por encima de un elemento (:hover) o hacer un clic.

- Animaciones (@keyframes): Funcionan como un libreto de cine con una línea de tiempo compleja. Permiten controlar detalladamente lo que pasa en los puntos intermedios usando porcentajes (ejemplo: al 0% el elemento es pequeño, al 50% gira y cambia de color, y al 100% se hace grande). No necesitan que el usuario interactúe para arrancar y pueden repetirse infinitamente.

- Casos de uso en la Experiencia de Usuario (UX):

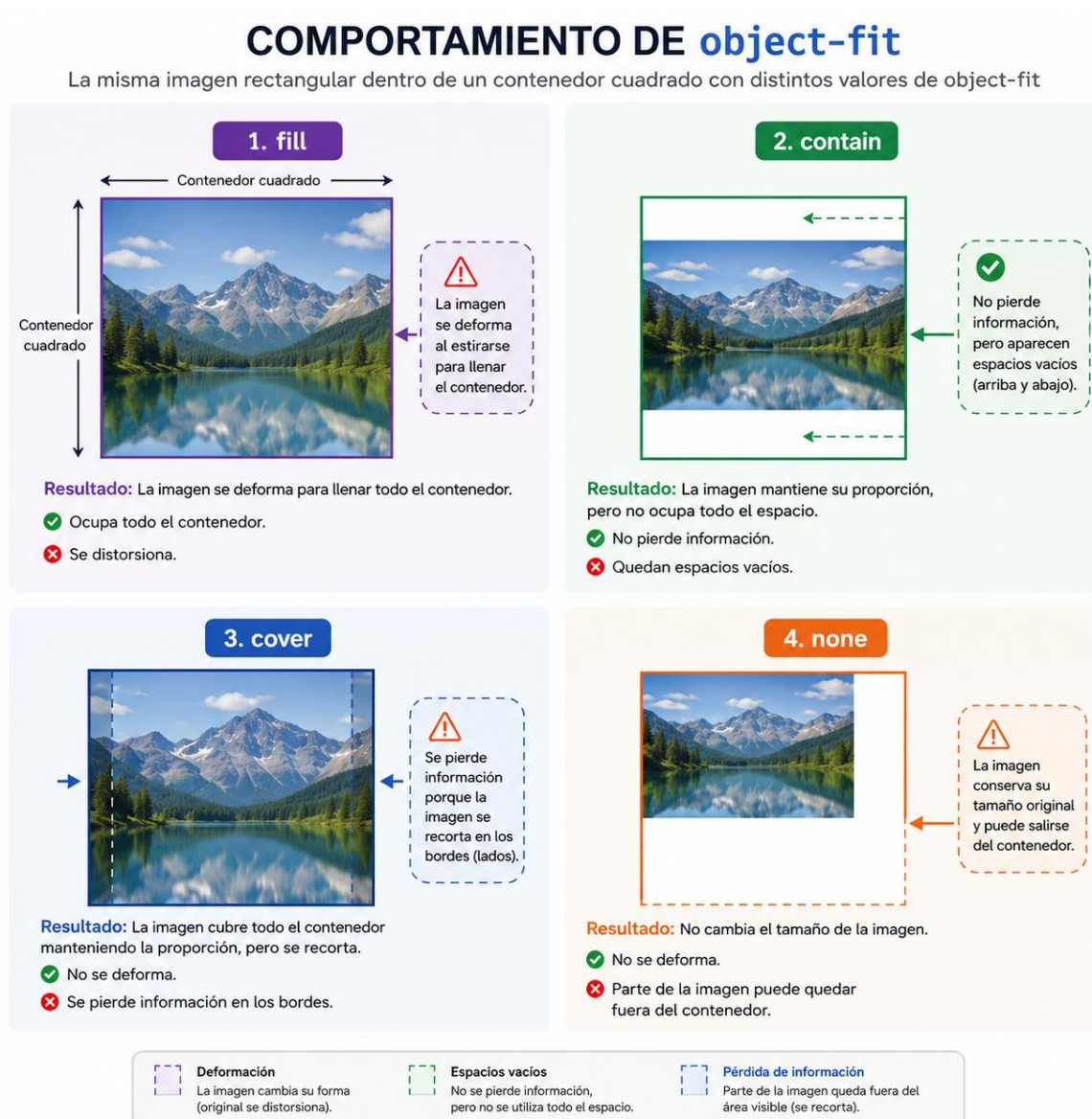
- Se prioriza una transición para dar respuestas inmediatas a lo que hace el usuario (micro interacciones), como cambiar suavemente el color de un botón al pasar el mouse para que la interfaz se sienta fluida y responsiva.

- Es estrictamente necesaria una animación con @keyframes cuando un elemento debe moverse o cambiar por sí mismo de forma continua y sin intervención del usuario, siendo el ejemplo perfecto los círculos de "Cargando..." (*spinners*) que giran en bucle mientras abre una página.



4. Infografía del Comportamiento de object-fit Realiza un cuadro comparativo visual que muestre la misma imagen rectangular de un paisaje dentro de un contenedor cuadrado, aplicando cuatro valores distintos: fill, contain, cover y none. Acota visualmente dónde se pierde información o dónde se deforma la imagen.

R/=



5. Esquema de Superposición Dinámica (z-index y Contextos de Apilamiento) Genera una representación en 3D (perspectiva isométrica o capas separadas flotantes) que explique visualmente cómo se comportan las capas en la pantalla cuando se usan diferentes valores de z-index combinados con position: relative y position: absolute

- **El Eje Z (Profundidad):** Controla el orden de superposición de los elementos en la pantalla (qué se dibuja adelante y qué se dibuja atrás). Un valor numérico mayor se coloca por encima de uno menor (por ejemplo, z-index: 10 tapaná a z-index: 2).

- **Relación con la propiedad position:** El z-index no tiene ningún efecto si el elemento tiene la posición por defecto (position: static). Requiere obligatoriamente un tipo de posicionamiento activo, como position: relative o position: absolute, para poder alterar su profundidad.

- **Contexto de Apilamiento (Stacking Context):** Es la jerarquía que agrupa las capas. Si un elemento padre tiene un nivel de prioridad bajo frente a sus elementos vecinos, ninguno de sus elementos hijos podrá pasar por encima de esos vecinos, sin importar qué tan alto sea el z-index del hijo. El hijo queda limitado al nivel de su contenedor.

LO VEMOS USUALMENTE EN POP-UPS, EN VENTANAS EMERGENTES, BARRA DE NAVEGACION, O PUBLICIDAD QUE SALDA EN LA PANTALLA QUITANDO EL CONTENIDO.

SUPERPOSICIÓN DINÁMICA: z-index y contextos de apilamiento

Las capas en la pantalla se apilan como una pila de libros: la que tiene mayor z-index queda más arriba.



EJEMPLOS FÁCILES

1 Elementos con position: relative

Se quedan en su lugar normal, pero su z-index define cuál se ve encima si se superponen.



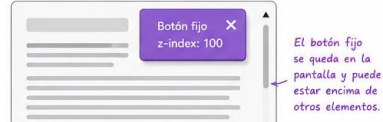
2 Elementos con position: absolute

Se posicionan respecto a su contenedor (el padre más cercano con posición).



3 Elementos con position: fixed

Se quedan fijos en la pantalla, sin importar el scroll, y su z-index decide si están encima de todo.



REGLA RÁPIDA

Mayor z-index = más arriba. | Mismo z-index = gana el que está después en el HTML. | Un nuevo contexto de apilamiento aísla a sus hijos.

¿Qué crea un nuevo contexto?

position: fixed, transform, opacity < 1, filter, will-change, entre otros.



